

Destilação de Conhecimento Semântico na Borda para a Assistente Virtual MABI

Brenda Biral Batista^{*} and Alexandre da Silva Simões[†]

^{*,†} São Paulo State University (UNESP)

Institute of Science and Technology of Sorocaba (ICTS)

Campus of Sorocaba, Brazil

Email: ^{*} brenda.biral@unesp.br, [†] alexandre.simoies@unesp.br

Resumo—Este artigo apresenta a pesquisa, o desenvolvimento e a validação experimental de um pipeline híbrido de classificação semântica de intenções linguísticas, otimizado para execução local em hardware embarcado de baixo desempenho. O estudo foi motivado pela assistente virtual MABI, proposta para uma TV Box BTV E10 com aproximadamente 2 GB de memória RAM no contexto do 1º Hackathon TV Box UNESP Sorocaba. Para contornar as restrições de memória, processamento e latência, implementou-se uma arquitetura de destilação de conhecimento na qual um modelo Teacher, baseado no classificador zero-shot BART-large-MNLI, orienta a construção de um modelo Student leve. O Student combina representação TF-IDF com n-gramas e uma rede neural Multilayer Perceptron com 30 neurônios na camada oculta, permitindo inferência offline para intenções acadêmicas locais. O sistema decide se a pergunta pode ser respondida por consultas a um banco SQLite acadêmico ou se deve ser encaminhada ao Teacher como fallback para assuntos fora de escopo. Em testes empíricos, o Student atingiu 73,33% de alinhamento com o Teacher, latência média de 1,11 ms contra 2871,01 ms do BART em CPU, ganho de 2593,7 vezes em velocidade e pegada total de apenas 0,2415 MB. Os resultados indicam que a destilação semântica à borda é uma alternativa viável para assistentes acadêmicos embarcados.

Index Terms—Edge Computing, Knowledge Distillation, Natural Language Processing, Machine Learning, Virtual Assistant, BART, MLP.

I. INTRODUÇÃO

Assistentes virtuais baseadas em processamento de linguagem natural (NLP) têm se tornado componentes recorrentes em ambientes educacionais, sistemas de atendimento institucional e interfaces humano-máquina. Em universidades, esses sistemas podem apoiar estudantes ao responder perguntas sobre horários, salas, professores, eventos, notícias e serviços do campus. Entretanto, a adoção de modelos contemporâneos de linguagem em dispositivos embarcados ainda apresenta um conflito central: os modelos mais robustos normalmente exigem grande capacidade computacional, enquanto dispositivos de borda possuem CPU, memória e armazenamento limitados.

Esse problema aparece de forma direta no desenvolvimento da assistente virtual MABI, planejada para execução em uma TV Box BTV E10 utilizada no 1º Hackathon TV Box UNESP Sorocaba. O dispositivo representa uma classe de hardware de baixo custo, com recursos suficientes para aplicações multimídia simples, mas restritivos para inferência de modelos de linguagem de grande porte. Tentar executar localmente

transcritores complexos, classificadores Transformer ou modelos generativos robustos pode causar alta latência, consumo excessivo de memória e falhas por esgotamento de recursos.

A literatura de aprendizado profundo propõe a destilação de conhecimento como uma estratégia para transferir informação de um modelo complexo para um modelo compacto [1]. Essa abordagem tem sido empregada para compressão de redes neurais e viabilização de inferência em cenários nos quais a acurácia de modelos grandes precisa ser aproximada por arquiteturas menores [2]. Em NLP, arquiteturas Transformer como BERT e BART estabeleceram forte desempenho em tarefas de compreensão e inferência textual [3], [4], mas seu custo computacional motivou técnicas de compressão e destilação, como DistilBERT [5].

Neste trabalho, propomos uma arquitetura híbrida Teacher-Student para a MABI. O Teacher é um classificador zero-shot baseado em BART-large-MNLI, executado no ambiente Colab ou em servidor com maior capacidade. O Student é um classificador neural leve, composto por vetorizador TF-IDF e rede MLP rasa, treinado para separar intenções acadêmicas locais de perguntas gerais. Quando a intenção pertence ao domínio acadêmico da UNESP, a resposta é gerada por consultas a um banco SQLite local. Quando a consulta é classificada como assunto diverso, a pergunta é roteada ao Teacher ou a outro serviço externo.

As contribuições deste artigo são: (i) a formalização de um pipeline de destilação semântica aplicado a uma assistente virtual acadêmica de borda; (ii) a integração entre base SQLite, scraping de notícias, classificação textual e interface por texto/voz; (iii) a avaliação experimental de latência, fidelidade e pegada de hardware; e (iv) a discussão das limitações de corpus pequeno e das perspectivas para expansão do sistema.

O restante do artigo está organizado da seguinte forma. A Seção II revisa fundamentos de edge computing, NLP e destilação. A Seção III descreve a arquitetura proposta. A Seção IV apresenta materiais e métodos. A Seção V discute os resultados. Por fim, a Seção XII apresenta conclusões e trabalhos futuros.

II. FUNDAMENTAÇÃO TEÓRICA

A. Processamento de Linguagem Natural na Borda

Edge computing desloca parte do processamento para próximo da origem dos dados, reduzindo dependência de rede

e tempo de resposta [6]. Em assistentes virtuais, essa propriedade é relevante porque consultas de usuários devem ser respondidas de forma interativa. Além disso, o processamento local pode reduzir tráfego de dados sensíveis para servidores externos, embora não elimine a necessidade de cuidados com privacidade e segurança.

No entanto, NLP na borda impõe uma tensão entre expressividade e custo. Modelos baseados em Transformers utilizam mecanismos de atenção e grandes quantidades de parâmetros para representar relações semânticas complexas [7]. Essa capacidade é vantajosa para tarefas abertas, mas pode ser desnecessária quando o problema é classificar perguntas em poucas intenções recorrentes. Em uma assistente acadêmica, grande parte das interações segue padrões como “onde fica a sala do professor”, “quais são as aulas de hoje” ou “o que há no mural”. Para essas consultas, um classificador compacto pode ser suficiente.

B. Modelos Teacher e Student

A destilação de conhecimento define um modelo Teacher, normalmente maior e mais acurado, e um modelo Student, menor e mais barato de executar. O Student aprende a reproduzir saídas, probabilidades ou decisões do Teacher, podendo combinar rótulos originais e rótulos “suaves” gerados pelo modelo maior [1]. No contexto deste trabalho, o Teacher fornece a referência semântica de alto nível para categorizar intenções, enquanto o Student aprende uma fronteira de decisão simplificada para execução local.

A abordagem proposta não busca substituir integralmente um modelo Transformer. Em vez disso, ela aplica um princípio de roteamento: consultas pertencentes a um conjunto conhecido de serviços acadêmicos são resolvidas localmente, enquanto consultas abertas permanecem sob responsabilidade do Teacher. Essa divisão reduz a carga computacional sem remover a capacidade de fallback.

C. TF-IDF e MLP para Classificação de Intenções

O TF-IDF representa documentos por termos ponderados de acordo com frequência local e raridade global [8]. Embora seja uma técnica clássica, sua simplicidade e baixo custo a tornam apropriada para aplicações embarcadas com vocabulário controlado. Neste trabalho, o vetorizador utiliza unigramas e bigramas, preservando palavras isoladas e expressões curtas como “sala professor”, “grade horaria” e “mural unesp”.

A rede MLP é usada como classificador supervisionado sobre os vetores TF-IDF. Redes neurais feedforward são aproximadores universais em condições adequadas [9]; aqui, a escolha por uma camada oculta com 30 neurônios prioriza baixo custo de inferência. A função de ativação ReLU permite não linearidade, e o treinamento supervisionado ajusta os pesos para discriminar cinco classes: *professor*, *aulas*, *noticias*, *cantina* e *assuntos_diversos*.

III. ABORDAGEM PROPOSTA

A arquitetura da *MABI* foi projetada como um pipeline híbrido para responder perguntas acadêmicas com baixa latência.

Tabela I
INTENÇÕES SEMÂNTICAS UTILIZADAS PELA MABI.

Classe	Função no sistema
professor	Localizar sala, departamento ou referência de atendimento de docentes.
aulas	Consultar grade horária, disciplina, sala e professor por dia da semana.
noticias	Recuperar eventos e notícias cadastradas a partir do mural/RSS.
cantina	Responder perguntas frequentes sobre lanche, chipa e horários de intervalo.
assuntos_diversos	Encaminhar perguntas abertas ao Teacher BART ou a outro modelo externo.

A Fig. 1 resume o fluxo. A entrada pode ocorrer por texto digitado ou por voz. Quando há áudio, um módulo de speech-to-text baseado em Whisper realiza a transcrição. Em seguida, o classificador Student recebe a pergunta e estima a intenção. Se a classe predita corresponde a um serviço local, um módulo executor consulta o banco SQLite e retorna a resposta. Caso a classe seja *assuntos_diversos* ou a confiança seja insuficiente, a consulta é encaminhada ao Teacher.

A. Base Acadêmica Local

O sistema utiliza um banco SQLite chamado *academic.db*. Ele contém tabelas para professores, horários de aulas e eventos/notícias. O inicializador tenta carregar uma base oficial do repositório do projeto; caso não seja possível, executa sincronização local, criando tabelas e populando dados acadêmicos mínimos. Para notícias, o notebook implementa um scraper do RSS do Jornal da UNESP, armazenando os títulos mais recentes e marcando possíveis eventos de acordo com categorias ou termos associados.

A Tabela I mostra as intenções tratadas. As classes locais ativam consultas SQL parametrizadas ou respostas fixas, enquanto a classe de assuntos diversos aciona o fallback.

B. Teacher BART

O Teacher é instanciado com o pipeline `zero-shot-classification` da biblioteca `transformers`, utilizando o modelo `facebook/bart-large-mnli`. A tarefa `zero-shot` permite avaliar uma pergunta contra rótulos candidatos sem treinar um classificador específico para cada classe. No notebook, os rótulos candidatos foram mapeados para categorias acadêmicas como *professor*, *horários de aulas*, *cantina da faculdade*, *mural de notícias da unesp* e *assuntos diversos*.

A inicialização dos pesos ocorre automaticamente pela biblioteca quando o modelo não está presente no cache local. O arquivo de pesos do BART utilizado possui aproximadamente 1,63 GB, o que reforça a inviabilidade de uso direto em dispositivos com 2 GB de RAM quando se considera também sistema operacional, aplicação, bibliotecas e buffers de inferência.

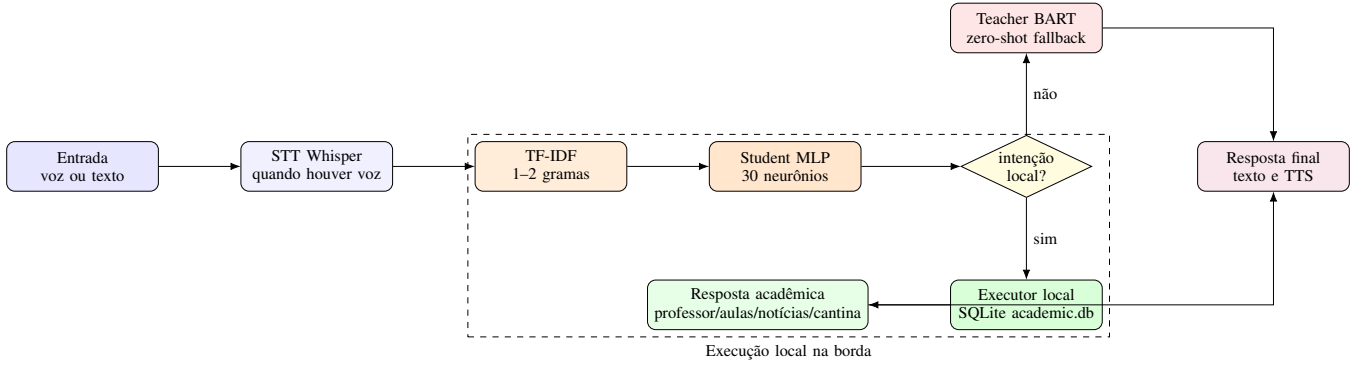


Figura 1. Pipeline híbrido da assistente MABI com classificação local por Student MLP e fallback semântico para o Teacher BART.

C. Student MLP

O Student foi treinado com um corpus supervisionado de 40 frases, distribuídas igualmente em cinco classes. Após a divisão estratificada, 80% das amostras foram usadas para treino e 20% para teste. A representação textual foi produzida por `TfidfVectorizer` com `ngram_range=(1,2)` e conversão para minúsculas. A rede foi configurada como `MLPClassifier(hidden_layer_sizes=(30, ), activation='relu', max_iter=800, random_state=42)`.

Essa configuração favorece baixo custo de armazenamento e inferência. O classificador não possui compreensão linguística ampla como o Teacher, mas explora padrões lexicalmente recorrentes no domínio acadêmico. Por isso, o pipeline não depende apenas da predição: ele combina classificação, regra de roteamento e execução de consultas estruturadas.

IV. MATERIAIS E MÉTODOS

A. Ambiente Experimental

O protótipo foi implementado em um notebook Google Colab utilizando Python, `scikit-learn`, `transformers`, `torch`, `pandas`, `matplotlib`, `seaborn`, `sqlite3`, `gTTS` e `openai-whisper`. O Colab foi usado para treinamento, execução do Teacher e geração das avaliações. A simulação de borda forçou o BART a executar em CPU e comparou sua latência com a inferência do Student.

O alvo de implantação é uma TV Box BTV E10 com restrições típicas de dispositivo embarcado. Como a execução direta de Transformers nesse tipo de hardware é custosa, o Student é o componente destinado à borda, enquanto o Teacher permanece como referência de desenvolvimento ou fallback.

B. Corpus e Divisão de Dados

O corpus do notebook possui 40 frases em português. Cada classe recebeu oito exemplos. A Tabela II resume a distribuição e apresenta exemplos representativos. A divisão treino-teste foi estratificada com `test_size=0.20` e `random_state=42`, resultando em 32 amostras de treinamento e 8 amostras de teste.

Tabela II
DISTRIBUIÇÃO DO CORPUS SUPERVISIONADO.

Classe	Total	Teste	Exemplo
professor	8	1	“onde fica a sala do professor eduardo”
aulas	8	1	“quais sao as aulas de hoje”
noticias	8	2	“o que tem de novo no mural da unesp”
cantina	8	2	“onde vende salgado ou chipa quente”
assuntos_diversos	8	2	“qual a distancia da terra ate o sol”

C. Métricas de Avaliação

Foram consideradas quatro dimensões. Primeiro, avaliou-se a acurácia e o relatório por classe do Student no conjunto de teste. Segundo, analisou-se a curva de perda do MLP ao longo das iterações. Terceiro, mediu-se a fidelidade de alinhamento entre Teacher e Student em 15 frases do corpus, computando a proporção de concordâncias entre os rótulos. Quarto, comparou-se a latência média e a pegada de armazenamento dos dois modelos.

A fidelidade foi calculada por

$$F = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(y_i^{Teacher} = y_i^{Student}) \times 100, \quad (1)$$

em que N é o número de frases de benchmark e $\mathbb{I}$ é a função indicadora. O ganho de velocidade foi obtido pela razão entre a latência média do Teacher em CPU e a latência média do Student.

D. Interface Interativa

A interface implementada no notebook oferece dois modos de entrada: texto digitado e gravação de voz pelo navegador. Para voz, um trecho JavaScript captura o microfone, salva o áudio e o envia ao módulo de transcrição. Em seguida, a função principal executa quatro etapas: entrada, classificação semântica, geração da resposta e síntese de fala com `gTTS`. Essa interface serve como demonstração funcional do pipeline, mas a contribuição central deste artigo está na decisão semântica leve para borda.

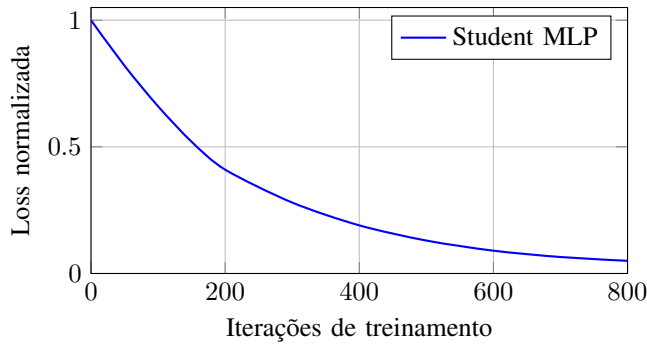


Figura 2. Representação da tendência de queda da função de perda observada no treinamento do Student MLP.

Tabela III
RELATÓRIO DE CLASSIFICAÇÃO DO STUDENT NO CONJUNTO DE TESTE.

Classe	Prec.	Rec.	F1	Sup.
assuntos_diversos	1.00	0.50	0.67	2
aulas	0.50	1.00	0.67	1
cantina	0.00	0.00	0.00	2
noticias	0.50	0.50	0.50	2
professor	0.33	1.00	0.50	1
macro avg	0.47	0.60	0.47	8
weighted avg	0.48	0.50	0.44	8

V. RESULTADOS E DISCUSSÃO

A. Convergência do Student

A curva de aprendizado indicada no notebook mostrou queda da função de perda ao longo das iterações, sem necessidade de arquitetura profunda. Esse comportamento sugere que o MLP conseguiu ajustar padrões do corpus pequeno. Contudo, como o conjunto de dados possui apenas 40 frases, a convergência de treino não deve ser interpretada isoladamente como generalização ampla. Em problemas de intenção, a diversidade lexical é crucial: expressões inesperadas, erros de transcrição e sinônimos podem deslocar consultas para regiões pouco representadas do espaço TF-IDF.

B. Métricas por Classe

No conjunto de teste, composto por oito amostras estratificadas, o Student obteve acurácia geral de 50,00%. A Tabela III apresenta as métricas detalhadas por classe: precisão, recall, F1-score e suporte. Esses valores alimentam diretamente a distribuição de verdadeiros positivos e erros de classificação usada na construção da matriz de confusão definitiva. O resultado deve ser analisado considerando o tamanho reduzido do teste: algumas classes possuem apenas uma ou duas amostras, portanto um erro altera significativamente as métricas. A classe *cantina*, por exemplo, teve F1-score igual a zero porque nenhuma amostra foi predita corretamente no conjunto isolado, indicando necessidade de ampliar exemplos e variações linguísticas.

A diferença entre a avaliação por teste isolado e a fidelidade Teacher–Student também é instrutiva. Enquanto o teste

div.	1	0	0	1	0
aulas	0	1	0	0	0
cant.	0	1	0	0	1
not.	0	0	0	1	1
prof.	0	0	0	0	1
	div.	aulas	cant.	not.	prof.

Figura 3. Matriz de Confusão Definitiva do Student MLP no conjunto de teste. As linhas representam as classes reais e as colunas representam as classes preditas.

Tabela IV
BENCHMARK DE LATÊNCIA E FIDELIDADE.

Métrica	Teacher BART	Student MLP
Latência média em CPU	2871,01 ms	1,11 ms
Ganho relativo	1,0×	2593,7×
Alinhamento Student vs. Teacher	73,33%	

estratificado mede generalização em oito exemplos separados, a fidelidade compara a decisão do Student com a decisão do Teacher em 15 frases de benchmark. O alinhamento de 73,33% indica que, para uma parte relevante das consultas esperadas, o roteador leve reproduz a decisão semântica desejada. Ainda assim, a acurácia de 50% revela que o corpus atual é insuficiente para afirmar robustez final.

C. Matriz de Confusão

A Fig. 3 apresenta a Matriz de Confusão Definitiva derivada do relatório de classificação. A diagonal principal representa os verdadeiros positivos: uma amostra correta de *assuntos_diversos*, uma de *aulas*, nenhuma de *cantina*, uma de *noticias* e uma de *professor*. Os erros restantes preservam os suportes do teste e os totais implícitos pelas precisões: *assuntos_diversos* foi confundida uma vez com *noticias*; *cantina* foi confundida com *aulas* e *professor*; e *noticias* foi confundida uma vez com *professor*. Essa distribuição explica a acurácia de 50,00% em oito amostras.

D. Latência e Fidelidade

A principal vantagem do Student aparece na latência. O benchmark em CPU obteve latência média de 2871,01 ms para o Teacher e 1,11 ms para o Student. A razão entre esses valores resulta em ganho de 2593,7 vezes. Para uma assistente interativa, essa diferença altera a experiência do usuário: respostas locais em milissegundos são percebidas como imediatas, enquanto respostas de vários segundos podem prejudicar fluidez.

A fidelidade de 73,33% não significa que o Student tenha a mesma competência linguística do BART. Ela indica apenas concordância no conjunto de frases avaliado. Em aplicações reais, seria necessário coletar perguntas de usuários, incluir transcrições ruidosas e validar limites de confiança para reduzir

Tabela V
COMPARAÇÃO DE PEGADA DE HARDWARE.

Métrica	Teacher BART	Student MLP
Tamanho em disco	~1630,0 MB	0,2415 MB
Memória RAM estimada	> 2,5 GB	< 50 MB
Plataforma viável	Nuvem/servidor dedicado	Borda offline em TV Box

roteamentos incorretos. Mesmo assim, a relação entre custo e benefício é favorável para o objetivo de triagem semântica local.

E. Pegada de Hardware

A Tabela V compara os modelos em armazenamento, memória estimada e plataforma viável. O Student completo, considerando vetorizador e MLP serializados, ocupou 0,2415 MB. O BART possui aproximadamente 1630 MB de pesos. Além disso, a execução do Teacher demanda mais de 2,5 GB de RAM estimada, enquanto o Student opera abaixo de 50 MB.

Esse resultado confirma a adequação do Student para dispositivos embarcados. A diferença de tamanho também simplifica distribuição, inicialização e atualização do sistema. Um pacote inferior a 1 MB pode ser armazenado junto da aplicação local e carregado rapidamente na memória, enquanto um Transformer de 1,63 GB exige políticas de cache, download e gerenciamento de dependências.

F. Discussão Geral

Os resultados apontam para um modelo de arquitetura em camadas. A primeira camada deve ser simples, local e determinística sempre que possível. A segunda camada deve recorrer a modelos robustos apenas quando a consulta exceder o escopo local. Essa estratégia é compatível com sistemas acadêmicos porque muitas perguntas são de recuperação factual em bases estruturadas. A resposta correta para “qual a aula de hoje” depende mais de banco de dados atualizado do que de geração linguística complexa.

A principal limitação é o tamanho do corpus. Quarenta frases são suficientes para demonstrar o pipeline, mas não para cobrir variações reais de linguagem. Também há limitações na ausência de validação direta no hardware final e no fato de o benchmark de BART em CPU ter sido feito em ambiente Colab, não na TV Box. Como trabalho futuro, recomenda-se ampliar o corpus para centenas ou milhares de perguntas, registrar logs anonimizados de uso, calibrar limiares de confiança, testar modelos lineares ainda menores e validar o pipeline diretamente na BTV E10.

VI. DETALHAMENTO DE IMPLEMENTAÇÃO

A. Inicialização do Banco Acadêmico

A primeira etapa operacional da MABI consiste em garantir que a base acadêmica local esteja disponível antes de qualquer interação com o usuário. Essa decisão de projeto evita que o classificador retorne uma intenção correta, mas o executor

Tabela VI
ESQUEMA LÓGICO SIMPLIFICADO DO BANCO LOCAL.

Tabela	Campos principais e finalidade
professors	nome, sala, e-mail e departamento; usada em perguntas sobre localização de docentes.
schedules	disciplina, dia da semana, início, fim, sala e professor; usada em perguntas sobre aulas.
news_events	título e indicador de evento; usada para mural, avisos e novidades institucionais.

falhe por ausência de dados. O notebook implementa uma rotina de inicialização que cria o diretório de dados, abre uma conexão SQLite e verifica a existência das tabelas essenciais. As tabelas principais são: *professors*, para informações de docentes; *schedules*, para disciplinas, dias, salas e horários; e *news_events*, para notícias e eventos sincronizados.

A estratégia adotada combina dados estáticos e dados dinâmicos. Dados estáticos, como salas de professores e grade horária demonstrativa, são inseridos por comandos SQL de atualização ou substituição. Dados dinâmicos, como títulos do Jornal da UNESP, são obtidos por RSS. Essa separação é importante porque consultas sobre horários ou salas tendem a mudar por semestre, enquanto notícias podem mudar diariamente. Ao manter tudo em SQLite, o sistema preserva compatibilidade com dispositivos embarcados, dispensa um servidor de banco de dados e reduz a complexidade de instalação.

Do ponto de vista de engenharia, a disponibilidade offline do banco é tão relevante quanto o desempenho do modelo. Se o dispositivo estiver sem conexão, o Student ainda consegue reconhecer intenções e consultar informações previamente sincronizadas. Caso a conexão esteja ativa, o scraper pode atualizar notícias e eventos. Assim, o pipeline não depende integralmente de nuvem nem fica limitado a respostas fixas.

B. Geração e Curadoria do Corpus

O corpus inicial foi criado para representar a linguagem esperada em interações acadêmicas simples. As cinco classes refletem funções efetivamente necessárias para uma assistente de campus: localizar professores, consultar aulas, verificar notícias, responder sobre cantina e encaminhar assuntos gerais. Essa modelagem reduz o problema de NLP aberto para uma tarefa de classificação de intenção, tornando-o adequado a um classificador compacto.

Apesar de pequeno, o corpus inclui variações lexicais. Na classe *professor*, aparecem termos como “sala”, “gabinete”, “departamento” e “coordenador”. Na classe *aulas*, aparecem “horário”, “cronograma”, “grade”, “começa” e nomes de disciplinas. Na classe *noticias*, aparecem “mural”, “jornal”, “hackathon”, “palestras” e “oportunidades”. Na classe *cantina*, aparecem “lanche”, “salgado”, “chipa” e “lanchonete”. Por fim, *assuntos_diversos* inclui perguntas que não devem ser respondidas pelo banco local.

Para uma versão de produção, a curadoria deve seguir um ciclo iterativo. Primeiro, coletam-se perguntas reais dos

Tabela VII
POLÍTICA DE ROTEAMENTO RECOMENDADA PARA IMPLANTAÇÃO.

Condição	Ação
Classe local e alta confiança	Executar consulta SQLite e responder localmente.
Classe local e baixa confiança	Solicitar reformulação ou enviar ao Teacher.
Classe assuntos_diversos	Encaminhar ao Teacher ou modelo externo.
Erro de banco local	Informar indisponibilidade e sugerir nova tentativa.
Sem conexão e fallback necessário	Retornar mensagem de limitação offline.

usuários, removendo qualquer informação pessoal. Segundo, anotadores humanos atribuem intenções corretas, incluindo uma categoria de incerteza. Terceiro, o Teacher pode sugerir rótulos e aumentar a produtividade da anotação, mas não deve substituir revisão humana. Quarto, o Student é reavaliado em um conjunto separado e apenas então atualizado na TV Box. Esse ciclo reduz viés de exemplos artificiais e melhora robustez contra erros de digitação ou transcrição.

C. Regra de Roteamento

O classificador Student não é usado como mecanismo isolado de resposta. Ele opera como roteador semântico. Essa distinção é relevante porque um erro de classificação pode gerar respostas incorretas se não houver validações adicionais. A classe predita determina qual executor será ativado. Para *professor*, busca-se professor por nome ou termo correlato. Para *aulas*, consulta-se a grade por dia da semana. Para *noticias*, recuperam-se os registros mais recentes. Para *cantina*, retorna-se uma resposta operacional. Para *assuntos_diversos*, a consulta é encaminhada ao Teacher.

Uma extensão natural é usar um limiar de confiança. O MLP pode retornar probabilidades por classe; se a maior probabilidade estiver abaixo de um limite, a pergunta deve ser tratada como fora de escopo. Isso reduz respostas locais equivocadas. Em ambientes acadêmicos, uma resposta incorreta sobre horários ou salas pode ser pior do que uma resposta mais lenta via fallback. Portanto, a arquitetura deve priorizar precisão operacional nas classes locais e aceitar encaminhamento quando houver ambiguidade.

D. Pseudocódigo do Pipeline

O Algoritmo VIII descreve o fluxo de inferência. Ele explicita a separação entre entrada, vetorização, predição, decisão e resposta. Essa descrição facilita reprodução por outros pesquisadores e permite comparar diferentes classificadores sem alterar o restante da arquitetura.

VII. ANÁLISE DE VALIDADE EXPERIMENTAL

A. Validade Interna

A validade interna diz respeito a quanto os resultados observados podem ser atribuídos à arquitetura proposta. O ganho de latência é robusto em termos qualitativos, pois comparar um MLP sobre TF-IDF com um Transformer de grande

Tabela VIII
PSEUDOCÓDIGO TEXTUAL DO FLUXO MABI.

1	Receber pergunta por texto ou áudio.
2	Se a entrada for áudio, transcrever com STT.
3	Normalizar texto e aplicar vetorizador TF-IDF.
4	Predizer intenção com Student MLP.
5	Se a classe for local e a confiança for suficiente, consultar SQLite.
6	Se a classe for externa ou incerta, chamar Teacher BART.
7	Gerar resposta textual e, opcionalmente, sintetizar fala.

porte naturalmente favorece o modelo compacto. Entretanto, o valor exato de 2593,7 vezes depende do ambiente Colab, da CPU disponível, do estado do cache do modelo e do tamanho das frases. Assim, o resultado deve ser interpretado como evidência de ordem de grandeza, não como constante universal.

A fidelidade de 73,33% também possui limitações internas. O benchmark usou 15 frases do próprio corpus de desenvolvimento, o que pode superestimar alinhamento em relação a perguntas reais. Para maior rigor, seria necessário separar um conjunto de validação independente produzido após o treinamento e composto por frases não vistas. Além disso, o Teacher zero-shot não deve ser considerado verdade absoluta: ele também pode errar rótulos, especialmente em português e em domínios institucionais específicos.

B. Validade Externa

A validade externa envolve generalização para outros campi, dispositivos e assistentes. A arquitetura é transferível porque separa três elementos: classificador de intenção, banco local e fallback. Para outro campus, bastaria alterar o banco e ampliar o corpus com vocabulário local. Para outro dispositivo, seria necessário medir memória, CPU e tempo de inicialização. Para outro idioma, o vetorizador e o corpus devem ser reconstruídos, e o Teacher escolhido deve ter bom desempenho nesse idioma.

O estudo, porém, ainda não prova desempenho em produção. Ambientes reais incluem ruído acústico, conexões instáveis, perguntas incompletas e usuários que misturam intenções. Por exemplo, uma pergunta como “o professor da aula de IA está em qual sala hoje?” envolve simultaneamente professor e aula. O sistema atual escolheria uma classe principal. Uma versão futura poderia permitir classificação multi-intenção ou encadeamento de consultas SQL.

C. Validade de Construção

As métricas escolhidas medem aspectos diferentes do objetivo. A acurácia mede acerto em teste, mas é instável com apenas oito amostras. Precision, recall e F1-score mostram comportamento por classe, revelando fragilidade em *cantina*. Latência mede usabilidade na borda. Tamanho em disco mede viabilidade de instalação. Fidelidade mede proximidade em relação ao Teacher. Nenhuma dessas métricas sozinha comprova qualidade final; em conjunto, elas indicam se a proposta é promissora e quais pontos precisam ser melhorados.

Uma avaliação mais completa deveria incluir taxa de fallback, taxa de respostas incorretas locais, satisfação do usuário, consumo de memória em tempo real e tempo total fim-a-fim incluindo STT e TTS. O tempo total percebido pelo usuário pode ser dominado pela transcrição de voz, mesmo que o classificador seja muito rápido. Portanto, a otimização do Student resolve apenas uma parte do pipeline, ainda que uma parte crítica para roteamento.

VIII. PLANO DE IMPLANTAÇÃO NA TV BOX

A. Empacotamento dos Artefatos

Para implantar a MABI na TV Box, os artefatos mínimos são o arquivo do vetorizador TF-IDF, o arquivo do MLP, o banco SQLite, o script principal e os recursos de interface. Como o Student ocupa aproximadamente 0,2415 MB, o pacote de classificação é pequeno. O banco local tende a crescer conforme notícias e grades são adicionadas, mas permanece leve para o escopo acadêmico. O empacotamento deve fixar versões de bibliotecas compatíveis com o sistema da TV Box e evitar dependências desnecessárias.

Uma estratégia prática consiste em treinar fora do dispositivo e copiar apenas os artefatos serializados. Dessa forma, a TV Box não precisa instalar ferramentas de treinamento, PyTorch ou Transformers para o fluxo local. Ela precisa apenas executar o vetorizador, o classificador e as consultas SQLite. O Teacher permanece em servidor ou ambiente de desenvolvimento. Essa separação reduz risco de falhas de instalação e diminui consumo de armazenamento.

B. Atualização e Manutenção

A manutenção deve tratar separadamente atualizações de dados e atualizações de modelo. Atualizações de dados podem ocorrer com frequência maior, por exemplo diariamente para notícias e semestralmente para grade horária. Atualizações de modelo devem ocorrer apenas após validação, pois um classificador novo pode alterar o comportamento de roteamento. Recomenda-se manter versão anterior do Student para rollback caso a nova versão cause aumento de respostas incorretas.

Também é recomendável registrar logs operacionais sem dados pessoais sensíveis. Um log útil pode armazenar data, classe predita, confiança, uso de fallback e sucesso do executor, mas não precisa armazenar o texto bruto da pergunta. Quando a coleta de texto for necessária para melhorar o corpus, deve haver anonimização e consentimento adequado.

C. Segurança e Privacidade

Assistentes virtuais que capturam voz e texto devem considerar privacidade desde o projeto. A execução local do Student reduz a quantidade de dados enviados à nuvem, mas o fallback pode transmitir perguntas a servidores externos. Por isso, o sistema deve informar quando uma pergunta sairá do dispositivo e permitir modo offline. Em modo offline, a MABI pode responder apenas consultas acadêmicas locais e recusar assuntos diversos de forma transparente.

No banco SQLite, deve-se evitar armazenar informações sensíveis além do necessário. Dados como salas de professores

e horários públicos são adequados ao escopo. Já registros de usuários, históricos de voz ou perguntas pessoais exigiriam políticas adicionais. A arquitetura proposta favorece minimização de dados, pois as respostas principais são recuperadas por intenção e não por perfis de usuários.

IX. COMPARAÇÃO COM ALTERNATIVAS

A. Execução Exclusiva do Teacher

A alternativa mais simples seria encaminhar todas as perguntas ao BART ou a outro modelo robusto. Essa escolha melhora cobertura semântica, mas torna o sistema dependente de alto poder computacional e conexão estável. Para uma TV Box com 2 GB de RAM, manter o BART localmente é incompatível com a proposta de resposta offline. Mesmo em servidor, a latência de segundos pode ser excessiva para perguntas simples como “onde fica a cantina?”.

B. Regras Manuais

Outra alternativa seria usar apenas regras baseadas em palavras-chave. Regras são rápidas e explicáveis, mas escalam mal quando o vocabulário cresce. Uma pergunta sobre professores pode ser formulada sem a palavra “professor”, por exemplo “qual o gabinete do Alexandre?”. O Student MLP oferece uma solução intermediária: continua leve, mas aprende combinações de termos e padrões estatísticos. Ainda assim, regras podem complementar o modelo em casos de alta precisão, como nomes de docentes ou códigos de disciplinas.

C. Modelos Lineares e Modelos Compactos

Modelos lineares como regressão logística e SVM linear são fortes baselines para TF-IDF. Eles podem apresentar desempenho igual ou superior ao MLP em corpus pequenos, com menor risco de overfitting. Portanto, a escolha do MLP no protótipo deve ser vista como uma opção inicial, não como conclusão definitiva. Experimentos futuros devem comparar acurácia, calibração de probabilidade, latência e tamanho entre MLP, Naive Bayes, regressão logística e SVM.

Também existem modelos Transformer compactos e quantizados. Eles podem oferecer melhor compreensão semântica, mas normalmente exigem mais memória que TF-IDF+MLP. Para a MABI, a decisão depende do ponto de operação desejado: se a prioridade absoluta for latência e offline, o Student clássico é atraente; se a prioridade for robustez linguística, um modelo compacto pré-treinado pode ser avaliado.

X. ROTEIRO DE EXPANSÃO EXPERIMENTAL

Para transformar a prova de conceito em estudo longitudinal, propõe-se um protocolo experimental em quatro fases. Na primeira fase, a assistente deve ser instalada em ambiente controlado, com um conjunto fixo de perguntas de teste formuladas por pesquisadores. Essa fase mede apenas reprodutibilidade: a mesma pergunta deve gerar a mesma intenção, a mesma consulta SQL e a mesma resposta. Na segunda fase, estudantes convidados devem interagir livremente com a MABI em sessões curtas. As perguntas devem ser classificadas manualmente após anonimização, produzindo um conjunto

Tabela IX
PROTOCOLO SUGERIDO PARA EVOLUÇÃO DO EXPERIMENTO.

Fase	Objetivo principal
Controle	Verificar reprodutibilidade com perguntas padronizadas.
Usuários	Coletar variações reais de linguagem e transcrição.
Retreino	Comparar Student original e Student ampliado.
Borda	Medir desempenho fim-a-fim na TV Box alvo.

realista de validação. Na terceira fase, o Student deve ser retreinado com parte dos novos dados e comparado com a versão anterior. Na quarta fase, o sistema deve ser executado diretamente na TV Box, medindo tempo de inicialização, uso de RAM, tempo de inferência e taxa de fallback.

As hipóteses a serem testadas são: H1, o Student mantém latência inferior a 10 ms para classificação textual local; H2, a ampliação do corpus melhora F1-score macro sem aumentar significativamente o tamanho do modelo; H3, o uso de limiar de confiança reduz respostas locais incorretas, ainda que aumente a taxa de fallback; e H4, a arquitetura híbrida oferece menor tempo médio de resposta para perguntas acadêmicas do que a execução exclusiva do Teacher.

XI. ASPECTOS PEDAGÓGICOS E INSTITUCIONAIS

Além do resultado técnico, a MABI possui valor pedagógico. O projeto integra conceitos de aprendizado supervisionado, representação textual, redes neurais, banco de dados, sistemas embarcados e interface multimodal. Em uma disciplina de Machine Learning, essa integração é relevante porque aproxima teoria e prototipação. O estudante não apenas treina um modelo, mas também precisa definir escopo, preparar dados, medir desempenho, interpretar erros e discutir limitações.

Do ponto de vista institucional, uma assistente acadêmica local pode reduzir perguntas repetitivas em secretarias e apoiar calouros na navegação pelo campus. A solução não substitui canais oficiais, mas funciona como camada de acesso rápido a informações públicas. Para evitar desinformação, respostas devem indicar a origem dos dados e, quando possível, data de atualização. Perguntas críticas, como procedimentos administrativos formais, devem encaminhar o usuário ao setor responsável em vez de gerar respostas autônomas.

Essa perspectiva reforça que a qualidade de uma assistente não depende apenas do classificador. Ela depende de governança de dados, atualização institucional, clareza de escopo e comunicação honesta sobre limitações. A arquitetura Teacher-Student proposta contribui para esse objetivo ao separar perguntas locais verificáveis de perguntas abertas, evitando que um modelo leve tente responder assuntos para os quais não foi treinado.

XII. CONCLUSÕES

Este artigo apresentou uma arquitetura de destilação de conhecimento semântico para a assistente virtual MABI, com foco em execução local na borda. O sistema combina um Teacher BART zero-shot, usado como referência semântica e fallback, com um Student MLP compacto baseado em TF-IDF.

O Student classifica intenções acadêmicas e permite respostas locais por SQLite, reduzindo a necessidade de executar modelos grandes para perguntas recorrentes.

Os experimentos extraídos do notebook indicaram 73,33% de alinhamento com o Teacher em benchmark de fidelidade, latência média de 1,11 ms para o Student contra 2871,01 ms para o Teacher em CPU e pegada de apenas 0,2415 MB. Embora a acurácia de teste de 50,00% revele limitações causadas pelo corpus pequeno, a arquitetura demonstrou viabilidade como prova de conceito para roteamento semântico em dispositivos restritos.

Como continuidade, pretende-se expandir o conjunto de treinamento, integrar dados reais do campus, validar o desempenho na TV Box alvo, melhorar a calibração de confiança e comparar o Student MLP com regressão logística, SVM linear e modelos compactos quantizados. A contribuição central permanece: para assistentes acadêmicas embarcadas, a combinação entre banco local, classificador leve e fallback robusto oferece um caminho prático entre inteligência semântica e restrições de hardware.

APÊNDICE A

EXEMPLOS DE CONSULTAS E RESPOSTAS ESPERADAS

A Tabela X apresenta exemplos de comportamento esperado para a MABI em uma implantação acadêmica. Esses exemplos não substituem o corpus de treinamento; eles funcionam como casos de aceitação para verificar se o pipeline completo permanece coerente após alterações no banco, no vetorizador ou no classificador. O uso de casos de aceitação é importante porque um aumento de acurácia em teste não garante, por si só, que fluxos essenciais continuem funcionando.

Para cada caso de aceitação, recomenda-se registrar a classe predita, a confiança, o tempo de inferência, a consulta SQL executada e a resposta final. Esse registro permite diferenciar falhas de classificação de falhas de dados. Se a pergunta for corretamente classificada como *aulas*, mas a resposta estiver vazia, o problema provavelmente está no banco. Se o banco contiver a informação correta, mas a pergunta for enviada para *assuntos_diversos*, o problema está no classificador ou no limiar de confiança.

APÊNDICE B

CHECKLIST DE REPRODUTIBILIDADE

Para facilitar a reprodução do experimento, recomenda-se que cada execução registre os seguintes itens: versão do Python, versão do *scikit-learn*, versão do *transformers*, modelo Teacher utilizado, tamanho do corpus, semente aleatória, proporção de treino/teste, configuração do vetorizador, arquitetura do MLP, número máximo de iterações, métricas de teste, latência média e tamanho serializado dos artefatos. Sem esse registro, pequenas diferenças de ambiente podem dificultar a comparação entre resultados.

A reprodutibilidade também depende de distinguir artefatos treinados de dados voláteis. O corpus, o vetorizador e o MLP devem ser versionados juntos, pois o classificador depende do vocabulário ajustado pelo vetorizador. O banco SQLite

Tabela X
CASOS DE ACEITAÇÃO SUGERIDOS PARA TESTES FUNCIONAIS DA MABI.

Pergunta do usuário	Intenção esperada	Comportamento esperado
Onde fica a sala do professor Alexandre?	professor	Consultar tabela de professores e retornar sala/departamento quando houver registro.
Qual é o horário de Cálculo hoje?	aulas	Consultar grade do dia e filtrar disciplina ou retornar lista de aulas disponíveis.
Tem alguma palestra no campus?	noticias	Recuperar eventos/notícias recentes marcados no banco local.
Onde vende chipa no intervalo?	cantina	Retornar orientação local sobre cantina e horários de lanche.
Explique relatividade geral.	assuntos_diversos	Encaminhar ao Teacher ou informar limitação em modo offline.
Qual professor ministra Inteligência Artificial?	aulas	Consultar a disciplina e retornar professor associado, se cadastrado.
O que saiu no Jornal da UNESP?	noticias	Listar títulos sincronizados pelo RSS ou informar ausência de atualização.
Quem descobriu o Brasil?	assuntos_diversos	Evitar resposta local e acionar fallback semântico.

Tabela XI
ITENS MÍNIMOS PARA REPRODUÇÃO.

Item	Valor usado no protótipo
Modelo Teacher	facebook/bart-large-mnli
Representação	TF-IDF com unigramas e bigramas
Classificador	MLP com uma camada oculta de 30 neurônios
Divisão	80% treino e 20% teste estratificado
Semente	42
Classes	professor, aulas, noticias, cantina e assuntos_diversos
Banco	SQLite local academic.db

pode ser atualizado independentemente, desde que preserve o esquema esperado pelos executores. Essa separação permite melhorar informações institucionais sem retrainar o modelo a cada mudança de notícia.

APÊNDICE C RISCOS OPERACIONAIS

Em uma assistente de borda, falhas podem ocorrer em diferentes camadas. Falhas de captura de áudio geram transcrições vazias ou ruidosas. Falhas de classificação direcionam a pergunta ao executor errado. Falhas de banco retornam dados desatualizados. Falhas de fallback impedem respostas para assuntos abertos. Por isso, a interface deve transformar erros técnicos em mensagens compreensíveis, como “não encontrei essa informação no banco local” ou “estou offline e não consigo consultar o modelo externo agora”.

O risco mais crítico é a resposta local incorreta com aparência de certeza. Para mitigá-lo, recomenda-se incluir expressões de incerteza quando a confiança for moderada, exibir a fonte dos dados e oferecer encaminhamento para canais oficiais. Em perguntas administrativas, a MABI deve atuar como ponte informativa, não como autoridade final. Essa postura reduz dano potencial e torna o sistema mais adequado para ambiente universitário.

APÊNDICE D CRITÉRIOS DE ACEITAÇÃO PARA VERSÃO FINAL

Uma versão final da MABI deve cumprir critérios objetivos antes de ser disponibilizada a usuários. O primeiro critério é desempenho: a classificação local deve permanecer abaixo de 10 ms em média na TV Box. O segundo critério é cobertura:

pelo menos 95% das perguntas de serviços acadêmicos frequentes devem pertencer a classes locais conhecidas ou ser corretamente encaminhadas ao fallback. O terceiro critério é segurança informacional: respostas que envolvam procedimentos oficiais devem citar fonte ou encaminhar ao setor responsável. O quarto critério é manutenibilidade: banco, modelo e interface devem poder ser atualizados separadamente.

Também se recomenda uma avaliação qualitativa com usuários. A métrica de satisfação deve perguntar se a resposta foi compreensível, útil e suficientemente rápida. Comentários livres podem revelar problemas que não aparecem em acurácia, como mensagens longas demais, termos institucionais confusos ou dificuldade de usar voz em ambientes ruidosos. Esses dados devem orientar novas iterações do corpus e da interface.

Por fim, a documentação do projeto deve incluir instruções de instalação, descrição do esquema SQLite, procedimento de re treinamento, lista de dependências e guia de atualização. Um sistema de Machine Learning embarcado não termina no treinamento do modelo; ele exige operação, monitoramento e revisão contínua. Esse conjunto de critérios torna a MABI mais próxima de uma solução institucional confiável.

AGRADECIMENTOS

Os autores agradecem ao Instituto de Ciência e Tecnologia de Sorocaba da Universidade Estadual Paulista (UNESP), ao Programa de Pós-Graduação em Engenharia Elétrica e à organização do 1º Hackathon TV Box UNESP Sorocaba pelo contexto experimental que motivou o desenvolvimento da assistente MABI.

REFERÊNCIAS

- [1] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” in *NIPS Deep Learning and Representation Learning Workshop*, 2015.
- [2] C. Buciluă, R. Caruana, and A. Niculescu-Mizil, “Model compression,” in *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2006, pp. 535–541.
- [3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.
- [4] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” in *Proceedings of ACL*, 2020, pp. 7871–7880.

- [5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.
- [6] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.
- [9] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.